MLIR Part 1 - Introduction to MLIR

Stephen Diehl

LLVM is a powerful compiler infrastructure project that has revolutionized how we build programming languages and tools. At its core, LLVM provides a collection of modular compiler and toolchain technologies that can be used to develop frontend compilers for any programming language and generate optimized code for many target architectures.

MLIR is a newer project within the LLVM ecosystem, designed to address the challenges of modern hardware accelerators and machine learning frameworks. While LLVM IR is powerful, it operates at a relatively low level. MLIR extends these concepts to support multiple levels of abstraction.

Unfortunately, the modern compiler stack is often something that one has to learn from reading RFCs, papers, and mailing lists. With this tutorial, I'll try to give a concise overview of the modern compiler stack, how MLIR fits into it, and how to integrate it into the Python machine learning ecosystem.

Throughout this series we'll cover the following topics eventually leading up to compiling a small version of the GPT2 transformer model down to efficient GPU kernels.

- Part 1 - What is MLIR?
- Part 2 - Memory in MLIR
- Part 3 - Affine Dialect and OpenMP
- Part 4 - Linear Algebra and Linalg
- Part 5 - Neural Networks and Tensors
- Part 6 - e-graphs and Term Rewriting
- Part 7 - NVIDIA GPU Execution
- Part 8 - Transformer Architecture
- Part 9 - Superoptimizing Deep Learning

This tutorial assumes some knowledge of C++ and advanced Python, along with passing familiarity with NVIDIA CUDA, but should otherwise be self-contained.

Why should I care?

You probably shouldn't. Writing optimizing compilers is a very niche topic. Although it's one that now with LLVM + MLIR requires *a lot less* knowledge than it used to even a few years ago. So if you want to write a new AI framework or experimental programming language, it's a good time. It's not strictly neccessary to learn, but then again so few things in life are.

Why MLIR?

MLIR was designed to address the limitations of traditional compilers like LLVM and GCC, particularly when dealing with modern hardware accelerators and AI workloads. While LLVM excels at traditional CPU targets, it struggles with the diverse specialized architectures emerging in AI and machine learning.

Modern AI systems typically involve a mix of CPUs, GPUs, TPUs, and custom ASICs. MLIR's flexible architecture makes it easier to work across these different platforms, providing robust support for heterogeneous hardware.

The **dialect** system in MLIR can represent and optimize operations at various levels, from high-level ML tasks down to hardware-specific instructions. This means we can implement optimizations tailored to specific domains, making AI workloads run more efficiently.

Instead of needing separate compilers for each type of accelerator or framework, MLIR offers a unified infrastructure that can be extended for different use cases. Traditional compilers weren't built with AI in mind, but MLIR has first-class support for tensor operations, neural networks, transformers, and other ML constructs.

MLIR has become the go-to tech for specialized machine learning accelerators, finding applications in signal processing, quantum computing, homomorphic encryption, FPGAs, and custom silicon. Its ability to create domain-specific compilers is a game-changer for those "weird domains" that don't fit the traditional CPU and GPU mold. Plus, it can be easily embedded in other languages, allowing us to build domain-specific languages tailored for specific tasks—something we'll dive into in this blog.

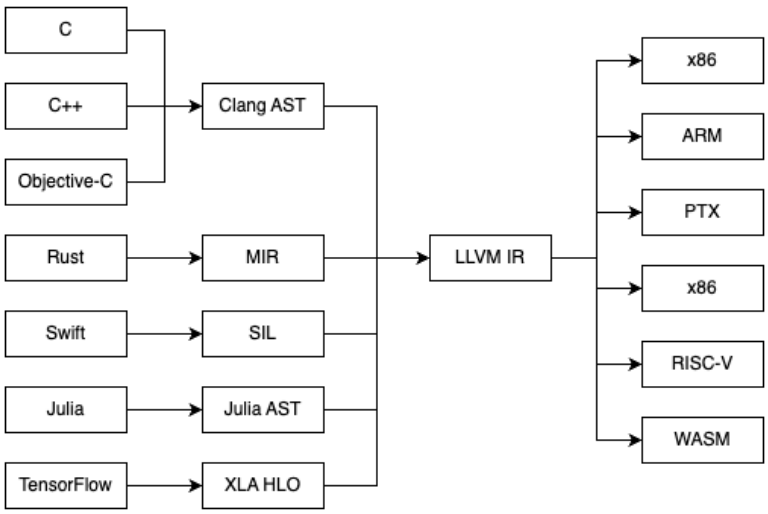
Key Components of the Modern Compiler Stack

1. **LLVM**: The heart of LLVM is its intermediate representation - a low-level programming language similar to assembly but with important high-level information preserved. This IR serves as a bridge between source languages and target architectures.
2. **MLIR**: MLIR is a new "multi-level" intermediate representation that is more expressive and can represent higher-level concepts like control flow, dataflow, and parallelism and be lowered into LLVM IR.
3. **Optimization Passes**: Both MLIR and LLVM provide a vast set of optimization passes that work on the IR level, allowing any language using LLVM to benefit from sophisticated optimizations. Most are written by grad students and PhDs on the latest and greatest research in compiler optimization. This is what makes LLVM such a powerful tool.
4. **E-Graphs** Equality saturation is a technique for building optimizing compilers using e-graphs, which originated from the automated theorem proving world; it operates by applying a set of rewrites using e-matching until the e-graph is saturated, continuing the saturation process until one of several halting conditions is met. After the rewriting phase, an optimal term is extracted from the e-graph based on a cost function, typically related to the size of the syntax tree or performance considerations, allowing for more efficient optimizations by exploring a broader set of potential transformations and selecting the best one based on defined criteria.

A modern compiler then weaves these components together to make up a pipeline.

1. Source code is parsed into a surface language.
2. Surface language is translated to a core language.
3. Core langauge can be optimized by core-to-core transformations (optionally using tools like e-graphs).
4. Core language is lowered to MLIR.
5. MLIR performs high-level optimizations
6. MLIR is lowered to LLVM IR
7. LLVM performs low-level optimizations and code generation

Many programmming langauges implement the "mid-level" IR as a way to bridge the gap between the source language and LLVM. For example, Rust, Swift, Tensorflow all follow this pattern. With MLIR we have can have a new generation of programming languages and frameworks that can reuse the same infrastructure and perhaps become more interoperable, in much the same way that LLVM has become the de facto backend for many programming languages.



Installing MLIR

Installing MLIR is a bit painful, it requires a few dependencies to be installed first.

```
# MacOS
brew install cmake ninja ccache

# Linux
apt-get install cmake ninja-build ccache
```

Then we can clone the LLVM repository and build MLIR from source. This can take a *long* time to build, I recommend having a cup of tea while you add entropy to the universe.

```
git clone https://github.com/llvm/llvm-project
mkdir llvm-project/build
```

```
cd llvm-project/build
cmake -G Ninja ../llvm \
   -DLLVM_ENABLE_PROJECTS=mlir \
   -DLLVM_BUILD_EXAMPLES=ON \
   -DLLVM_TARGETS_TO_BUILD="Native;ARM;X86" \
   -DCMAKE_BUILD_TYPE=Release \
   -DLLVM_ENABLE_ASSERTIONS=ON \
   -DCMAKE_C_COMPILER=clang -DCMAKE_CXX_COMPILER=clang++ \
   -DLLVM_CCACHE_BUILD=ON
cmake --build . --target check-mlir
cmake --build . --target install
```

Check the version of MLIR installed. This tutorial is using version 21.0.0.

```
mlir-opt --version
LLVM (http://llvm.org/):
  LLVM version 21.0.0git
  Optimized build with assertions.
```

At some point in the near future the Homebrew version of LLVM will be updated to include MLIR, at which point this will be a lot easier. If you're reading this at some point in a better future you can just try this and it might work.

```
brew install llvm
```

MLIR Concepts

MLIR is a programming language in its own right, it's very low-level but it contains many of the core primitives you might find in other languages. First some identifier conventions:

- `%` prefix: SSA values (e.g. `%result`)
- `@` prefix: Functions (e.g. `@fun`)
- `^` prefix: Basic blocks (e.g. `^bb0`)
- `#` prefix: Attribute aliases (e.g. `#map_1d_identity`)
- `x` delimiter: Used in shapes and between shape and type (e.g. `10xf32`)
- `:` and `->` are used to indicate the type of an operation or value (e.g. `%result: i32`)
- `!` prefix: Type aliases (e.g. `!avx_m128 = vector<4 x f32>`)
- `()` are used to represent arguments (e.g. `(%arg0, %arg1)`)
- `{}` are used to represent regions
- `//` is used for comments
- `<>` are used to indicate type parameters (e.g. `tensor<10xf32>`)

1. **Modules**: The top-level container for MLIR operations.

   ```
   module {
     // Operations
   }
   ```

2. **Functions**: A function is a collection of operations that are executed in a specific order.

   ```
   func.func @my_function(%arg0: i32, %arg1: i32) -> i32 {
     // Operations
   }
   ```

3. **Dialects**: Domain-specific sets of operations and types in MLIR.

   The high-level ones well use are:

   - `tensor`: This dialect provides operations for creating and manipulating multi-dimensional arrays (tensors), enabling high-level mathematical expressions and transformations in a side-effect-free manner.
   - `linalg`: The linear algebra (linalg) dialect offers a set of operations specifically designed for linear algebra computations, facilitating efficient implementations of matrix and vector operations.
   - `omp`: The OpenMP (omp) dialect includes operations that support parallel programming models, allowing developers to express parallelism in a way that is compatible with OpenMP standards.
   - `affine`: The affine dialect provides a framework for expressing affine operations and analyses, enabling optimizations that leverage the predictable behavior of affine expressions.
   - `gpu`: The GPU dialect is specialized for GPU programming, providing operations for parallel execution on heterogeneous architectures.

   The lower-level ones are:

   - `scf`: The structured control flow (scf) dialect provides operations that represent structured control flow constructs, such as loops and conditionals, ensuring a clear and maintainable flow of execution.
   - `func`: This dialect encompasses operations related to function definitions and calls, facilitating the organization and modularization of code through high-level function abstractions.
   - `memref`: The memref dialect is dedicated to memory reference operations, allowing for efficient management and manipulation of memory buffers, essential for performance-critical applications.
   - `index`: The index dialect specializes in operations for handling index computations, which are crucial for addressing elements in arrays and tensors, particularly in loop iterations and data access patterns.
   - `arith`: The arithmetic (arith) dialect contains fundamental mathematical operations for both integer and floating-point types, including basic arithmetic, bitwise operations, and comparisons, applicable to scalars, vectors, and tensors.

   Additionally we'll use some hardware specific dialects for targeting NVidia GPUs, but more on that later.

4. **Operations**: The basic unit of work in MLIR, similar to LLVM instructions but with an additional dialect namespace and possible type annotations.

   ```
   // Example of an MLIR operation
   %0 = "my_dialect.my_operation"(%arg0, %arg1) : (i32, i32) -> i32
   ```

   In this example, `my_dialect.my_operation` is an operation defined in the `my_dialect` dialect, and `%arg0` and `%arg1` are arguments of type `i32`. It returns a result of type `i32` bound to the variable `%0`.

   A concrete example would be the "arith.addf" operation, which adds two floating point numbers.

   ```
   %0 = arith.addf %arg0, %arg1 : f32
   ```

5. **Basic Blocks**: Basic blocks in MLIR are sequences of operations that execute in a linear fashion without any branching. Each basic block has a single entry point and can have multiple exit points, typically through control flow operations like branches or returns. Basic blocks are essential for structuring the flow of control in a program, allowing for clear and maintainable code organization. They enable the compiler to optimize the execution path and facilitate transformations such as inlining and loop unrolling. In MLIR, basic blocks are defined within functions and can be manipulated through various dialects to represent complex control flow constructs.

   ```
   ^bb1: // Label for the then block
   %then_result = arith.muli %result, 2 : i32
   return %then_result : i32
   ```

   Unike in LLVM, basic blocks can now also take arguments, which are passed in through the `^bb1(%result : i32)` syntax.

6. **Regions**: Regions are a way to group operations together. They are used to represent control flow constructs like loops and conditionals. Regions are grouped together in `{}` blocks and can take arguments.

   ```
   {
     ^bb1(%result : i32):
     %then_result = arith.muli %result, 2 : i32
     return %then_result : i32
   }
   ```

7. **Types**: A type is a classification that specifies which kind of value a variable can hold and what operations can be performed on it. Types help to enforce constraints on data, ensuring that operations are performed on compatible values.

   ```
   %result = arith.constant 1 : i32
   ```

   We can also defined type synonyms for convenience.

   ```
   !avx_m128 = vector<4 x f32>
   ```

8. **Passes**: Transformations that operate on the MLIR dialects, optimizing and lowering them into simpler constructs.

   These are arguments that can be passed to `mlir-opt` to transform the MLIR, the most common ones are:

   - `convert-func-to-llvm`: Convert function-like operations to LLVM dialect
   - `convert-math-to-llvm`: Convert math operations to LLVM dialect
   - `convert-index-to-llvm`: Convert index operations to LLVM dialect
   - `convert-scf-to-cf`: Convert structured control flow to CF dialect
   - `convert-cf-to-llvm`: Convert control flow to LLVM dialect
   - `convert-arith-to-llvm`: Convert arithmetic operations to LLVM dialect
   - `reconcile-unrealized-casts`: Reconcile unrealized casts
   - `convert-memref-to-llvm`: Convert memref operations to LLVM dialect
   - `convert-tensor-to-llvm`: Convert tensor operations to LLVM dialect
   - `convert-linalg-to-scf`: Convert linalg operations to `scf.for` loops
   - `convert-linalg-to-affine-loops`: Convert linalg operations to `affine.for` loops
   - `convert-omp-to-llvm`: Convert OpenMP operations to LLVM dialect
   - `convert-vector-to-llvm`: Convert vector operations to LLVM dialect

   There is also a generic `-convert-to-llvm` pass that will convert anything that can be converted from MLRI to LLVM IR. In practice we'll use more granular passes to convert specific dialects to LLVM.

   If you need more granular control over the passes you can also specify the pass names in a comma-separated list in a `--pass-pipeline` string, e.g., `--pass-pipeline="builtin.module(pass1,pass2)"`. The passes will be run sequentially in one group.

   Additionally there are several use flags for debugging pass transformation. The `--mlir-print-ir-after-all` flag prints the IR after each pass, while `--mlir-print-ir-after-change` and `--mlir-print-ir-after-failure` provide more specific output. When using any of these print-ir flags, including `--mlir-print-ir-tree-dir`, the IRs are written to files in a directory tree if you don't want to parse through the terminal stdout.

   Note: The order of the passes can be important, for example `convert-scf-to-cf` must come before `convert-cf-to-llvm`.

Creating and Using LLVM Modules

Let's create a simple LLVM module that returns 42 as an exit code, compile it to a shared library, and use it from Python. This example demonstrates the complete pipeline from LLVM IR to usable code.

1. Writing LLVM IR

First, create a file named `simple.ll` with this LLVM IR:

```
define i32 @main() {
    ret i32 42
}
```

As an analogue this program is equivalent to:

```
int main() {
    return 42;
}
```

2. Compiling to a Shared Object

We can compile this LLVM IR to a shared object using `clang`:

```
llc -filetype=obj --relocation-model=pic simple.ll -o simple.o
clang -shared -fPIC simple.o -o libsimple.so
clang simple.o -o simple # optionally create an executable
```

Now if you run the executable you should see the output `42` returned as the exit code to the process.

```
$ ./simple
$ echo $?
42
```

3. Using from Python

Now we can use this shared library `libsimple.so` from Python using `ctypes`:

```
import ctypes

module = ctypes.CDLL("./libsimple.so")

module.main.argtypes = []
module.main.restype = ctypes.c_int

print(module.main())
```

Running this you should see the Python logic successfully call the shared library and print the result.

```
$ python simple.py
42
```

MLIR Universe

Now for a function that returns 42, it's simple enough to write in LLVM. But non-trivial programs in LLVM is not so simple, it is indeed often too "low-level" for many applications and over the years many people have constructed higher-level abstractions on top of LLVM.

For example writing loops in LLVM is actually a bit annoying as you have to manually handle blocks, phi nodes, all for a basic loop. MLIR allows us to write high-level constructs directly and have them lowered down into LLVM in a single pass.

4. Writing MLIR

First, create a file named `simple.mlir` with this MLIR:

```
func.func @loop_add() -> (index) {
    %init = index.constant 0
    %lb = index.constant 0
    %ub = index.constant 10
    %step = index.constant 1

    %sum = scf.for %iv = %lb to %ub step %step iter_args(%acc = %init) -> (index) {
        %sum_next = arith.addi %acc, %iv : index
        scf.yield %sum_next : index
    }

    return %sum : index
}

func.func @main() -> i32 {
    %out = call @loop_add() : () -> index
    %out_i32 = arith.index_cast %out : index to i32
    func.return %out_i32 : i32
}
```

In C this is equivalent to:

```
int loop_add(int lb, int ub, int step) {
  int sum_0 = 0;
  int sum = sum_0;
  for (int iv = lb; iv < ub; iv += step) {
    sum_next = sum + iv;
    sum = sum_next;
  }
  return sum;
```

```
}

int main() {
  int out = loop_add(0, 10, 1);
  return out;
}
```

4. Lowering to LLVM Dialect

LLVM IR is based on Static Single Assignment, which means that every variable is assigned exactly once and every use of a variable must be dominated by its definition. In SSA, instead of variables being reassigned multiple times, each new "assignment" creates a new version of the variable, typically denoted with a numbered suffix (like `%1`, `%2`, etc.). This property makes many compiler optimizations simpler and more effective because the relationships between definitions and uses are explicit and unambiguous.

For example, in LLVM IR, you might see code like:

```
%1 = add i32 %a, %b
%2 = mul i32 %1, %c
```

Here, `%1` and `%2` are SSA values that can never be reassigned. When control flow requires a variable to have different values depending on the path taken (like after an if-statement), LLVM uses special "phi" nodes to merge the different possible values, maintaining the SSA property while handling dynamic program behavior.

In LLVM basic blocks are the basic unit of control flow, they are a sequence of instructions that are executed sequentially. A basic block has one entry and one exit point.

Phi nodes are special instructions in SSA-based intermediate representations that resolve multiple potential values from different control flow paths into a single value. They're essentially control-flow-aware multiplexers that select the appropriate value based on which path the program took to reach that point.

```
entry:
  br i1 %cond, label %then, label %else

then:
  %x.1 = add i32 1, 2
  br label %merge

else:
  %x.2 = mul i32 3, 4
  br label %merge

merge:
  %x.3 = phi i32 [ %x.1, %then ], [ %x.2, %else ]
  ret i32 %x.3
```

In this example, the phi node `%x.3` will take the value of `%x.1` (which is 3) if control flow came from the `then` block, or `%x.2` (which is 12) if it came from the `else` block. The syntax `[ value, predecessor_block ]` specifies which value to use depending on which block was previously executed.

We can lower the high-level MLIR to LLVM dialect using `mlir-opt`. Essentially reducing the loop constructs to the branches over blocks statements.

```
mlir-opt example.mlir \
  --convert-func-to-llvm \
  --convert-math-to-llvm \
  --convert-index-to-llvm \
  --convert-scf-to-cf \
  --convert-cf-to-llvm \
  --convert-arith-to-llvm \
  --reconcile-unrealized-casts \
  -o example_opt.mlir
```

This will produce the following MLIR:

```
module {
  llvm.func @loop_add() -> i64 {
    %0 = llvm.mlir.constant(0 : i64) : i64
    %1 = llvm.mlir.constant(0 : i64) : i64
    %2 = llvm.mlir.constant(10 : i64) : i64
    %3 = llvm.mlir.constant(1 : i64) : i64
    llvm.br ^bb1(%1, %0 : i64, i64)
  ^bb1(%4: i64, %5: i64):  // 2 preds: ^bb0, ^bb2
    %6 = llvm.icmp "slt" %4, %2 : i64
    llvm.cond_br %6, ^bb2, ^bb3
  ^bb2:  // pred: ^bb1
    %7 = llvm.add %5, %4 : i64
    %8 = llvm.add %4, %3 : i64
    llvm.br ^bb1(%8, %7 : i64, i64)
  ^bb3:  // pred: ^bb1
    llvm.return %5 : i64
  }
  llvm.func @main() -> i32 {
    %0 = llvm.call @loop_add() : () -> i64
    %1 = llvm.trunc %0 : i64 to i32
    llvm.return %1 : i32
  }
}
```

We can use the `mlir-cpu-runner` to run the `main` function directly from MLIR code.

```
mlir-cpu-runner -e main -entry-point-result=i32 simple_opt.mlir

# To use the runner utils (e.g. for debug printing)
# mlir-cpu-runner -e main -entry-point-result=i32 -shared-libs=/opt/homebrew/opt/llvm/lib/libmlir_runner_utils.dylib simple_opt.mlir
```

5. Compiling to a Shared Object

From there is a direct translation of the MLIR to LLVM IR using `mlir-translate`. We can compile this MLIR to a shared object using `mlir-translate`:

```
mlir-translate simple_opt.mlir -mlir-to-llvmir -o simple.ll
llc -filetype=obj --relocation-model=pic simple.ll -o simple.o
clang -shared -fPIC simple.o -o libsimple.so
```

The LLVM them compiles down into assembly (in this case ARM64 assembly).

```
_loop_add:
sub     sp, sp, #0x30
mov     x8, #0x0
mov     x9, x8
str     x9, [sp, #0x20]
str     x8, [sp, #0x28]
b       0x100003f10
ldr     x9, [sp, #0x20]
ldr     x8, [sp, #0x28]
str     x8, [sp, #0x8]
str     x9, [sp, #0x10]
subs    x9, x9, #0xa
str     x8, [sp, #0x18]
b.ge    0x100003f4c
b       0x100003f30
ldr     x9, [sp, #0x10]
ldr     x8, [sp, #0x8]
add     x8, x8, x9
add     x9, x9, #0x1
str     x9, [sp, #0x20]
str     x8, [sp, #0x28]
b       0x100003f10
ldr     x0, [sp, #0x18]
add     sp, sp, #0x30
ret
```

Standard MLIR Dialects

`llvm`

The `llvm` dialect is the dialect that represents LLVM IR. It is the lowest level dialect in the MLIR hierarchy, it directly passes through to LLVM IR.

`scf` and `cf`

All higher-level 'scf' structures compile into 'cf' constructs.

- `cf.br` - A branch to a basic block
- `cf.cond_br` - A conditional branch to a basic block
- `cf.switch` - A switch to a basic block

An unconditonal branch is a branch that always goes to the same basic block.

```
// A basic block
^bb0:
  cf.br ^bb1
```

A conditional branch is a branch that goes to a different basic block depending on a condition variable (here `%c`).

```
// A conditional branch to a basic block
^bb0:
  cf.cond_br %c, ^bb1, ^bb2
```

A switch is a branch that goes to a different basic block depending on a switch variable (here `%i`).

```
// A switch to a basic block
^bb0:
  cf.switch %i, ^bb1, ^bb2, ^bb3
```

For example we could put these constructs together to create a function that selects between two values based on a boolean flag returning one of the given arguments.

```
func.func @select(%a: i32, %b: i32, %flag: i1) -> i32 {
  cf.cond_br %flag, ^bb1(%a : i32), ^bb1(%b : i32)

^bb1(%x : i32) :
  return %x : i32
}
```

The most basic structured control flow construct is the `scf.if` construct.

```
scf.if %b  {
  // true region
} else {
  // false region
}
```

Like we did a above a for loop is a structured control flow construct, it has a single entry and a single exit point.

```
%lb = index.constant 0
%ub = index.constant 10
%step = index.constant 1

scf.for %iv = %lb to %ub step %step {
  // loop region
}
```

This would be lowered with the `convert-scf-to-cf` pass into a sequence of `cf.br` and `cf.cond_br` operations.

`arith` and `math`

The `arith` dialect includes basic operations like:

- Integer arithmetic: `addi`, `subi`, `muli`, `divsi` (signed), `divui` (unsigned)
- Floating point arithmetic: `addf`, `subf`, `mulf`, `divf`
- Comparisons: `cmpi` (integer), `cmpf` (float)
- Conversions: `extsi` (sign extend), `extui` (zero extend), `trunci`, `fptoui`, `fptosi`, `uitofp`, `sitofp`
- Bitwise operations: `andi`, `ori`, `xori`

Example of arithmetic operations:

```
func.func @arithmetic_example(%a: i32, %b: f32) -> i32 {
    // Integer addition
    %1 = arith.constant 42 : i32
    %2 = arith.addi %a, %1 : i32

    // Float to integer conversion
    %3 = arith.fptosi %b : f32 to i32

    // Final addition
    %4 = arith.addi %2, %3 : i32
    return %4 : i32
}
```

The `math` dialect provides more complex mathematical operations:

- Trigonometric: `sin`, `cos`, `tan`
- Exponential: `exp`, `exp2`, `log`, `log2`, `log10`
- Power functions: `pow`, `sqrt`
- Special functions: `erf`, `atan2`
- Constants: `constant`

Example using math operations:

```
func.func @math_example(%x: f32) -> f32 {
    // Calculate sin(x) * sqrt(x)
    %1 = math.sin %x : f32
    %2 = math.sqrt %x : f32
    %3 = arith.mulf %1, %2 : f32
    return %3 : f32
}
```

The passes `-convert-math-to-llvm` and `-convert-arith-to-llvm` will convert the math and arithmetic operations to basic LLVM operation equivalents.

`index`

The `index` dialect is specialized for handling index computations and is particularly important for array/tensor operations. An index type in MLIR is a platform-specific sized integer used for addressing and loop induction variables.

Key index operations include:

- `index.constant`: Create an index constant
- `index.add`: Add two indices
- `index.sub`: Subtract two indices
- `index.mul`: Multiply two indices
- `index.cmp`: Compare two indices
- `index.divs`: Signed division of indices
- `index.rems`: Signed remainder of indices

Index operations are used to compute the offset of an element in an array. For example, the following function computes the offset of an element in a 2D array. We can treat them as natural number operations as they are always non-negative.

```
func.func @compute_offset(%i: index, %j: index, %stride: index) -> index {
    // Compute row-major array offset: i * stride + j
    %1 = index.mul %i, %stride
    %2 = index.add %1, %j
    return %2 : index
}
```

The pass `-convert-index-to-llvm` will convert the index dialect to the LLVM dialect.

The index type is particularly useful when working with tensors and memory layouts, as it provides a natural way to express array indices and dimensions. It's used in conjunction with the `memref` and `tensor` dialects for array access patterns. Which we'll cover next.

External Resources

- MLIR Builtin Dialect
- MLIR: A Compiler Infrastructure for the End of Moore's Law
- MLIR: Scaling Compiler Infrastructure for Domain Specific Computation
- 2023 LLVM Dev Mtg - MLIR Is Not an ML Compiler, and Other Common Misconceptions
- A MLIR Dialect for Quantum Assembly Languages
- Building an End-to-End Toolchain for Fully Homomorphic Encryption with MLIR